Reichman University
Efi Arazi School of Computer Science
M.Sc. Program in Machine Learning and Data Science

Final Project Report

Permuted Dogs versus Cats

Student: Roy Rubin ID: 201312907

Advisors: Alon Oring (Reichman University)

July 9, 2026

Abstract

Image classifiers are usually trained and evaluated on images whose spatial layout is intact. This report studies how much binary Dogs-versus-Cats classification performance survives when that layout is disrupted by fixed tile-wise image permutations. Each image from the Kaggle Dogs vs Cats dataset is divided into a square grid, its tiles are rearranged by a deterministic permutation, and the same permutation is applied to both the training and validation images for a given run. The study treats this setting as an empirical computer-vision benchmark motivated by structured-permutation hardness, not as a cryptographic security experiment.

The experiments are organized into three parts. First, frozen ImageNet-1k-pretrained MobileNetV3-Small, DeiT-Tiny, and gMLP-S16 backbones are compared using trained binary classification heads over one unpermuted baseline and deterministic 4 $\times$ 4, 7 $\times$ 7, and 10 $\times$ 10 tile grids, each paired with easy, medium, and hard permutation presets. Part 1 also includes two reviewer-requested controls: full fine-tuning of the pretrained binary-head models and zero-shot evaluation of each native ImageNet-1k head by comparing summed dog-class and cat-class probabilities. Second, several configurable CNN-backbone improvement strategies are evaluated against matched baselines, including patch shuffle, regular image augmentation, mixed original/permuted training, a nonlinear MLP classification head, and two curriculum variants. Third, model-agnostic permutation hardness metrics are computed and joined to the saved configurable-CNN validation results; this part does not train additional models.

The study uses ImageNet-1k-only timm checkpoints for MobileNetV3-Small, DeiT-Tiny, and gMLP-S16. The expected behavior is that classification performance remains above chance under fixed deterministic permutations, but decreases as the grid becomes finer and the permutation becomes more disruptive. The findings are limited to fixed deterministic permutations, a single train/validation split, one main seed, frozen pretrained backbones, no independent test set, and exploratory hardness correlations over a small deterministic matrix whose primary analysis collapses the

repeated clean baseline to one row. Best validation accuracy is also optimistic because it selects the best epoch on the validation set. Therefore, the report provides preliminary evidence that useful dog/cat cues survive the tested transformations, but it does not show that the models reconstruct, invert, transfer across, or explicitly learn the original image layout.

1 Introduction

Image classification models are typically evaluated on data whose spatial structure is intact. This report asks how much binary Dogs-versus-Cats classification performance survives when that structure is deliberately disrupted by a fixed tile-wise permutation. Each input image is split into an $N \times N$ grid, the tiles are rearranged by a deterministic mapping, and the classifier must still distinguish dogs from cats on a held-out validation set transformed by the same target permutation.

The main research question is: *under a fixed supervised train/validation split, how do grid size, deterministic permutation preset, and pretrained visual representation affect validation accuracy after tile-wise image permutation?* The report also asks two secondary, testable questions. First, which configurable-CNN training or head modifications improve performance relative to the matched frozen-backbone baseline? Second, do deterministic, model-agnostic permutation hardness metrics correlate with the observed configurable-CNN best validation accuracy across the final experiment matrix?

The task is motivated by permuted-puzzle problems and cryptographic hardness [2], but the cryptographic connection is motivational rather than directly tested. The present image setting is a relaxed empirical analogue: it uses natural images rather than graphs of functions, applies coarse tile-wise rather than pixel-wise permutations, and evaluates supervised classification rather than cryptographic recovery or security. The results should therefore be interpreted as evidence about this computer-vision testbed, not as evidence about cryptographic hardness.

The study contributes a controlled, reproducible empirical benchmark for analyzing how fixed spatial rearrangements affect transfer-learning classifiers within this experimental matrix:

- It defines a deterministic Dogs-versus-Cats tile-wise permutation matrix over one unpermuted baseline and 4×4, 7×7, and 10×10 grids, each crossed with easy, medium, and hard presets.
- It compares three frozen ImageNet-1k-pretrained representation families: convolutional MobileNetV3-Small, attention-based DeiT-Tiny, and gMLP-S16.
- It evaluates six configurable-CNN improvement strategies against matched baselines: patch shuffle, standard image augmentation, mixed original/permuted training, a nonlinear MLP head, and two curricula.
- It quantifies each deterministic permutation with model-agnostic hardness metrics and relates those metrics to held-out validation accuracy.

The experiment sequence follows the three requested assignment parts. Part 1 trains model baselines on the one-tile/no-permutation baseline and deterministic tile-wise permutations at 4×4, 7×7, and 10×10 resolution. Part 2 tests improvement strategies for the same permuted setting, including patch shuffle, regular image augmentation, mixed original/permuted training, a larger configurable-CNN classification head, and two curricula. Part 3 computes permutation hardness metrics and joins them to validation accuracy so that image-independent permutation properties can be compared against model performance.

2 Related Works

2.1 Cryptographic and Permuted-Puzzle Motivation

The motivating proposal cites permuted puzzles as a source of cryptographic hardness, where fixed permutations can make structured objects difficult to recover or classify [2]. This report does not solve or evaluate a cryptographic task. Instead, it borrows the idea of structured permutation as a controlled way to disrupt spatial organization in images. The cryptographic motivation provides conceptual background, while the empirical claims are limited to supervised classification on the Dogs vs Cats benchmark under deterministic tile-wise permutations.

2.2 Spatial Ordering

Spatial ordering is a central source of information in images. Nearby pixels and patches usually belong to related object parts, textures, contours, or background regions, and many visual recognition methods exploit this structure either directly or through learned representations. Context-prediction and jigsaw-style work show that relative patch position and tile arrangement contain semantic signal [3, 12]; rotation prediction similarly demonstrates that image geometry can support representation learning [5]. The present report uses spatial reordering differently: the task is supervised Dogs-versus-Cats classification under a fixed deterministic tile-wise permutation, not reconstruction of the original image and not prediction of the permutation.

2.3 Convolutions

Convolutional neural networks are built around locality, weight sharing, and translation-related structure. These assumptions are well matched to ordinary natural images, where local textures, edges, and object parts often carry semantic information. MobileNetV3 is a compact convolutional family designed with efficient inverted residual blocks, squeeze-and-excitation, and architecture-search-informed components [10]. Tile-wise permutation disrupts many adjacency relations between neighboring image regions while preserving local content inside each tile, making a convolutional backbone a useful reference point for measuring how much local visual evidence survives the permutation.

2.4 Vision Transformers

Vision Transformers represent an image as a sequence of patch tokens and use self-attention to exchange information across the sequence [4]. DeiT extends this family with data-efficient ImageNet-scale training and distillation-aware design choices [15]. This patch-token view is relevant for tile-wise permutation because individual patches may remain informative even when global spatial arrangement is corrupted. At the same time, positional embeddings, patch size, attention depth, and pretraining all affect how a transformer responds to reordered image content.

2.5 Robustness, Augmentation, and Curricula

The CNN-backbone ablations connect to broader regularization and robustness ideas in deep learning [7]. Patch shuffle and regular image augmentations introduce additional stochastic variation during training; such augmentation is a standard way

to improve generalization when the training perturbations match useful invariances [13]. Mixed original/permuted training tests whether clean-layout examples help the classifier head fit cues that transfer to the permuted validation distribution. Curriculum learning variants test whether staged exposure to increasing corruption probability or increasing permutation difficulty improves optimization [1]. In this report, these methods are evaluated as practical interventions for a frozen pretrained configurable CNN backbone, not as general claims about robustness under all spatial corruptions.

2.6 MLPs and MLPs for Vision

Multilayer perceptrons are usually associated with channel-wise feature recombination, but vision MLP architectures also mix information across spatial positions. MLP-Mixer showed that patch-token features can be processed with alternating token-mixing and channel-mixing MLP layers [14]. gMLP uses spatial gating to mix token information without self-attention and has been shown to work on both language and vision tasks [11]. This makes gMLP-S16 a useful non-convolutional, non-attention comparison point for testing whether patch-level token mixing can classify fixed tile-permuted images.

3 Data Overview

The dataset is Kaggle's Dogs vs Cats image dataset. The labeled portion contains 25,000 RGB JPEG images, with 12,500 cat images and 12,500 dog images. Original image dimensions are heterogeneous: in the local dataset copy, widths range from 42 to 1050 pixels and heights range from 32 to 768 pixels, with 8,513 unique width-height pairs. The most common sizes are approximately 500×374 and 499×375, but the preprocessing pipeline resizes images before tiling so that all model inputs in a run have a consistent square resolution.

The supervised split is stratified with seed 42 and an 80/20 train/validation ratio. This gives 20,000 training images and 5,000 validation images. Both splits are exactly class-balanced: the training split contains 10,000 cats and 10,000 dogs, and the validation split contains 2,500 cats and 2,500 dogs. The validation split is held out from optimization and is not modified by training-only augmentations such as patch shuffle, regular augmentation, original/permuted sampling, or curriculum-stage sampling. Tables 6, 7, and 9 provide the detailed per-part reuse of this split in Section 5.

For the unpermuted setting, each image is processed as a normal dog or cat image and used as the benchmark for post-permutation performance. For the permuted setting, each image is divided into a square tile grid and the same deterministic tile-wise permutation is applied across images for a run. The final grid settings are one tile/no permutation, 16 tiles (4×4), 49 tiles (7×7), and 100 tiles (10×10). The one-tile/no-permutation baseline is repeated under the easy, medium, and hard labels only so that all parts use the same 4×3 grid-permutation matrix. Aggregate summaries should account for the fact that these three records correspond to the same unpermuted condition rather than three distinct permutations.

The easy, medium, and hard presets provide three controlled levels of spatial disruption at each nontrivial grid size. Easy performs a small number of local swaps and preserves most adjacency structure. Medium increases the fraction of swapped tiles and adds row and column shifts. Hard combines the largest swap fraction, broader swap distance, stronger shifts, and a global reversal. These

presets make it possible to compare models not only as the number of tiles increases, but also as the deterministic transformation becomes more disruptive at a fixed grid size. The results reported below describe this specific experimental design.

4 Methodology

The experimental work is organized into three assignment parts. Part 1 establishes baselines by training on the unpermuted task and on fixed deterministic tile-wise permutations. Part 2 reuses the Part 1 configurable-CNN baseline and trains improvement ablations on the same task family. Part 3 computes permutation descriptors and correlates them with the validation accuracy observed for the corresponding deterministic records.

Part 1 compares MobileNetV3-Small, DeiT-Tiny, and gMLP-S16. Part 2 and Part 3 use MobileNetV3-Small for the configurable-CNN experiments. Aggregation reports mean final validation accuracy, mean best validation accuracy, standard deviation across the three deterministic permutation labels at each tile count, and the number of runs. Validation accuracy is the main evaluation metric because the held-out validation set is class-balanced. Best validation accuracy is useful for comparing training behavior across runs, but it can be optimistic as a final performance estimate because it selects the best epoch using the validation set.

4.1 Models Used in the Baseline Comparison

MobileNetV3-Small is used as the convolutional baseline because it is compact and efficient while still exposing ImageNet-1k pretrained features [10]. DeiT-Tiny represents the transformer family in a small pretrained configuration, using patch embeddings, multi-head self-attention, and a classification token [15]. gMLP-S16 represents a non-convolutional, non-attention patch model in which spatial gating mixes information across patch tokens [11]. All three models are used as frozen pretrained feature extractors in Part 1, so the experiment compares fixed representations plus trained binary heads rather than full end-to-end adaptation.

4.2 Deterministic Permutation Functions

Every part uses one shared deterministic generator with three parameter presets: easy, medium, and hard. This makes comparisons across Parts 1, 2, and 3 reproducible and interpretable, because a given difficulty label refers to the same construction wherever it appears.

The generator is controlled by relative parameters: the fraction of tiles to swap, the maximum swap distance as a fraction of grid width, row and column shift fractions, and whether to apply a global reversal before the shifts and swaps. This matters because fixed operation counts are not comparable across grid sizes: two swaps affect 25% of the tiles in a 4×4 grid but only 4% of the tiles in a 10×10 grid. The easy preset therefore uses a small tile-count-scaled swap fraction and short local movement; medium increases the swap fraction and adds moderate row and column shifts; hard uses the largest swap fraction, broader movement radius, stronger shifts, and a global reversal. For the one-tile baseline, all three labels map to the unpermuted image; the labels are still saved so the experiment matrix remains exactly 4 grid settings by 3 presets.

Preset	Swap frac.	Max dist. frac.	Row shift	Col shift	Reversal
Easy	0.05	0.15	0.00	0.00	No
Medium	0.14	0.35	0.20	0.20	No
Hard	0.28	0.80	0.45	0.45	Yes

Table 1: Deterministic permutation preset parameters, as defined in `tile_permutations.py`.

4.3 Architecture Comparison and Frozen Backbones

To compare architectural inductive biases, the Part 1 pipeline uses three pretrained feature extractors: MobileNetV3-Small [10], DeiT-Tiny [15], and gMLP-S16 [11]. All three are loaded from timm with ImageNet-1k-only pretrained weights. In the main Part 1 transfer-learning regime, the ImageNet-pretrained backbone is frozen and only the final binary classification head is optimized. This setting evaluates pretrained representations plus a small trained classification head under the tested permutations; it does not test full representation learning under permutation. The control matrix adds an unfrozen pretrained binary-head condition. A second control preserves the native ImageNet-1k head and performs zero-shot Dogs-versus-Cats prediction by summing the pretrained dog-breed logits' softmax probabilities and comparing them with summed domestic-cat probabilities.

The three models differ in how they exchange information across the image. MobileNetV3-Small uses lightweight convolutions, giving it a local-neighborhood bias. DeiT-Tiny represents the image as patches and uses self-attention to let patches interact. gMLP-S16 also operates on image patches, but replaces attention with MLP layers and spatial gating for token mixing. The comparison is informative but not fully controlled: the models differ not only in inductive bias, but also in parameter count, patch/tokenization behavior, and feature geometry.

Model	Main operation	Inductive bias
MobileNetV3-Small	Convolution	Local texture and translation structure
DeiT-Tiny	Self-attention	Patch-to-patch interactions
gMLP-S16	Spatial-gating MLP	Patch and feature mixing without attention

Table 2: Architectural comparison of the three frozen feature extractors.

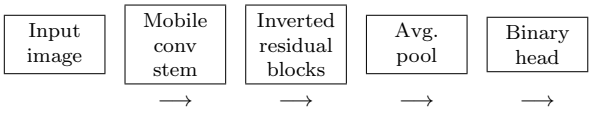

Figure 1: General MobileNetV3-Small architecture used in the convolutional experiments. The ImageNet-1k-pretrained backbone is frozen in the main transfer-learning runs; only the final binary head is optimized, except that the Part 2 MLP-head ablation replaces the linear head with a small nonlinear head.

Table 4 records additional implementation details for the three Part 1 backbone choices. All selected checkpoints are ImageNet-1k-only timm checkpoints; ImageNet-21k, JFT, and hybrid-pretrained checkpoints are rejected by the model factory. All three selected native classifiers expose a 1000-class ImageNet-1k output head before the Dogs-versus-Cats experiment replaces that head with a binary classifier. The code can freeze each selected backbone: in the binary-head baseline only the final `classifier` or `head` parameters remain trainable, while the zero-shot full-head control freezes all parameters and performs no optimization.

The model factory uses explicit timm model identifiers rather than framework defaults. This matters because it prevents accidental use of ImageNet-21k-to-1k, JFT, or hybrid checkpoints when comparing frozen pretrained representations.

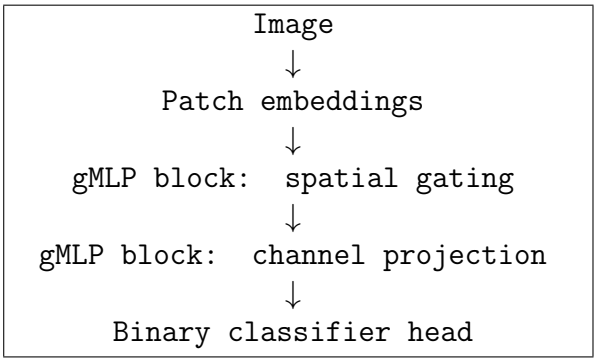

Figure 2: Simplified gMLP flow. Spatial gating exchanges information across patches, while channel projections recombine features within each patch representation.

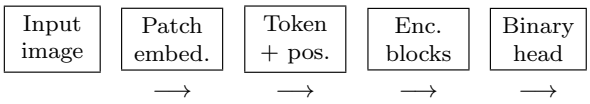

Figure 3: General DeiT-Tiny architecture used in the transformer experiments. The image is converted into patch tokens, combined with a classification token and positional embeddings, processed by transformer encoder blocks, and read out through a binary classification head.

4.4 Optimization and Hyperparameters

The original Part 1 baseline and all Part 2 training runs use ImageNet-pretrained backbones with the feature extractor frozen and only the classification head trainable. The added unfrozen Part 1 control keeps the pretrained initialization and binary head but allows all model parameters to update. The zero-shot full-head control performs no gradient updates and reports validation accuracy from the native ImageNet-1k classifier. For trained binary-head runs, the loss is cross-entropy over the two classes. The optimizer is AdamW with learning rate 3×10^{-4} and weight decay 1×10^{-4}. Runs use seed 42. In the executed Colab configuration, the batch size is 64, the dataloader uses 4 workers, and

Model	Learnable	Frozen	Total
MobileNetV3-Small	2,050	1,517,856	1,519,906
DeiT-Tiny	386	5,524,416	5,524,802
gMLP-S16	514	19,165,656	19,166,170

Table 3: Parameter counts for the frozen-backbone comparison with two output classes. "Learnable" counts only the classifier head optimized during training; "Frozen" counts pretrained parameters used only in the forward pass.

Model	Selected checkpoint	Native head	Freeze behavior
MobileNetV3-Small	`mobilenetv3_small_100`	`classifier`, 1000 classes	freeze all except `classifier`
DeiT-Tiny	`deit_tiny_patch16_224.fb_in1k`	`head`, 1000 classes	freeze all except `head`
gMLP-S16	`gmlp_s16_224.ra3_in1k`	`head`, 1000 classes	freeze all except `head`

Table 4: Pretraining, native classifier head, and freezing behavior for the selected Part 1 model checkpoints.

automatic mixed precision is enabled. The local configuration defaults to smaller smoke-test settings, so the reported results should be read as the Colab run matrix.

Part 1 trained binary-head runs train each model/permutation record for 10 epochs; the zero-shot full-head control uses 0 epochs. Part 2 non-curriculum ablations also train for 10 epochs. The two curriculum ablations train for 30 epochs, distributed across their stages as evenly as possible: corruption-probability curriculum uses four stages with probabilities 0.1, 0.3, 0.5, and 0.8, while permutation-difficulty curriculum starts from original images and progresses through 2×2, 3×3, 4×4, and, when needed, the final target grid. Part 3 does not train a model; it computes deterministic permutation metrics and joins them to the saved configurable-CNN accuracies.

Images are normalized with ImageNet mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. The standardized model input size is 224 pixels. Preprocessing may temporarily increase the square resize size to the smallest value divisible by the re-

quested grid size before tiling; for example, 10×10 tiling uses 230 before rearrangement. The final tensor is then center-cropped back to 224 for the timm backbones. Regular augmentation uses random rotation up to 15 degrees, color jitter with brightness/contrast/saturation 0.2 and hue 0.05, resize, tensor conversion, and normalization. Patch shuffle uses a random within-image patch permutation during training only.

4.5 Improvement Strategies

Part 2 evaluates training strategies against the regular Part 1 configurable-CNN baseline, which defaults to MobileNetV3-Small. The main distinction is whether a strategy adds stochastic training views, mixes clean and permuted inputs, changes the classifier head, or changes the order in which the model sees easy and hard inputs. In all cases, validation is performed on the held-out validation images with the target permutation and without training-only perturbations.

For the counts below, let B be the configured batch size and let $N_{\text{train}} = 20{,}000$ be the number of training images. The ablations do not append files to the dataset. Instead, they replace or resample the ordinary training view during the forward pass, so the stored increase is zero even when every processed example is augmented or curriculum-scheduled.

4.5.1 Regular Part 1 Baseline

Main idea. This is the reference point for Part 2. The configured CNN backbone is trained normally on the target permutation, with no extra augmentation, no curriculum, and the standard loss.

Reason. The baseline is necessary because every ablation must be judged against ordinary training on the same task. Without it, an improvement strategy might look useful simply because the target permutation is easier than another run.

Deeper dive. The baseline uses the same train and validation split as Part 1. For a permuted run, both training and validation images are transformed by the same fixed tile permutation for that run. It has one training stage and does not increase the dataset size. If x_i is an input image, P_π is the target tile permutation, and f_θ is the classifier, the baseline minimizes

$$\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \ell(f_\theta(P_\pi(x_i)), y_i) \, .$$

The number of added augmented images is 0 per batch and 0 per epoch; the number of processed training views remains B per batch and 20,000 per epoch.

4.5.2 Patch Shuffle

Main idea. Patch shuffle randomly rearranges smaller patches during training. The model therefore sees extra local spatial disruptions in addition to the fixed tile permutation.

Reason. This is relevant because the main task already tests whether class information survives spatial reordering. If extra random patch movement helps, it would suggest that tolerance to local layout disruption is useful for the tested permuted Dogs versus Cats setting.

Deeper dive. The target permutation is still the run-level task definition, but each training batch can also receive random patch shuffling. Let S_ρ denote a random within-image patch permutation sampled independently during training. The effective training view is

$$\tilde{x}_i = S_{\rho_i}(P_\pi(x_i)) \, ,$$

and the loss is computed on $\ell(f_\theta(\tilde{x}_i), y_i)$. In the implementation the shuffle probability is

1.0, so a batch of size B contains B shuffled views and an epoch contains 20,000 shuffled views. These views are not appended to the dataset; they replace the current batch examples. The validation loader does not use patch shuffle, so reported accuracy measures the fixed target validation permutation.

4.5.3 Regular Augmentations

Main idea. Regular augmentations apply standard image-level stochastic transformations during training while keeping the target validation permutation fixed.

Reason. This tests whether generic image augmentation helps the frozen CNN head fit dog-versus-cat cues that transfer to the target tile permutation, or whether the extra variation is mismatched to the spatial corruption used at validation time.

Deeper dive. The training set remains the same 20,000 images. For each example, an image-level transform E is sampled during training:

$$\tilde{x}_i = E(P_\pi(x_i)) \,.$$

The strategy has one stage and trains for 10 epochs. It stores no additional images; it produces one stochastic view per processed training example. Validation uses the deterministic target permutation without training-only augmentation.

4.5.4 Mixed Original/Permuted Training

Main idea. Mixed original/permuted training exposes the model to clean images and target-permuted images in the same run.

Reason. If clean images preserve class structure that is useful for the permuted task, mixing them into training may stabilize the classifier. If the clean examples pull the head away from the target validation distribution, the strategy may hurt.

Deeper dive. The ablation samples each training view as original with probability $p_{\text{original}} = 0.5$ and target-permuted otherwise:

$$\tilde{x}_i = \begin{cases} x_i, & z_i < p_{\text{original}}, \\ P_\pi(x_i), & z_i \geq p_{\text{original}}, \end{cases}$$

where $z_i \sim \text{Uniform}(0, 1)$. This gives an expected 10,000 clean views and 10,000 permuted views per epoch. The validation loader remains fully target-permuted, so the reported accuracy measures transfer from the mixed training distribution to the target permutation.

4.5.5 CNN MLP Head

Main idea. This ablation keeps the frozen configurable CNN backbone but replaces the linear classifier with a small MLP classification head.

Reason. The frozen backbone may still extract useful ImageNet features from permuted tiles, but the mapping from those features to dog/cat labels may be less linearly separable after spatial disruption. A small MLP head tests whether additional trainable capacity at the classifier stage improves the target task.

Deeper dive. The data, target permutations, validation loader, and 10-epoch non-curriculum schedule match the regular Part 1 baseline. The only intentional change is the classifier head, so improvements in this ablation can be interpreted as preliminary evidence that head capacity matters under the frozen-backbone setup.

4.5.6 Corruption-Probability Curriculum

Main idea. The model starts with mostly intact images and gradually sees the target permutation more often. The corruption probability increases across four stages.

Reason. Jumping immediately into a hard permutation may make optimization unstable. This curriculum tests whether gradual exposure lets the frozen-backbone classifier first fit dog-versus-cat cues and then adapt those cues to more frequent spatial corruption.

Deeper dive. Each stage uses the same 20,000 training images and target tile permutation, but changes the probability that the permutation is applied. For stage probability q_s, the sampled training view is

$$\tilde{x}_i = \begin{cases} P_\pi(x_i), & z_i < q_s, \\ x_i, & z_i \geq q_s, \end{cases} \qquad z_i \sim \text{Uniform}(0, 1).$$

The stages use $q_s \in \{0.1, 0.3, 0.5, 0.8\}$, so the expected number of permuted views per stage epoch is $q_s N_{\text{train}}$: 2,000, 6,000, 10,000, and 16,000 respectively. In a batch of size B, the expected number of permuted views is $q_s B$. This does not store a larger dataset; it changes the sampled training view. There is no separate validation set for each stage; validation always uses the fixed target validation loader. Curriculum runs train for 30 epochs in the final saved results.

4.5.7 Permutation-Difficulty Curriculum

Main idea. The model starts on easier spatial structure and moves toward the final harder permutation. Instead of changing probability, it changes the tile grid difficulty across stages.

Reason. Smaller or easier permutations preserve more of the original image structure. This experiment tests whether a model can learn useful class cues under mild disruption before facing the final, harder tile rearrangement.

Deeper dive. Each stage uses the same 20,000 training images but changes the tile grid used for training. If g_s is the stage grid size and π_s is the corresponding sampled or target permutation, then

$$\tilde{x}_i^{(s)} = \begin{cases} x_i, & g_s = 1, \\ P_{\pi_s}^{(g_s)}(x_i), & g_s > 1. \end{cases}$$

The schedule begins with original images and then progresses through 2×2, 3×3, and 4×4 permutations; for 7×7 and 10×10 target runs, it adds a final target-grid stage. Stage permutations use the same deterministic difficulty label as the target record. Each stage processes B scheduled-difficulty views per batch and 20,000 per stage epoch. The stored dataset size does not increase, validation remains the held-out loader for the final target run, and curriculum runs train for 30 epochs in the final saved results.

4.6 Tile Permutation Hardness Metrics

We quantify the difficulty induced by a tile permutation using three complementary, model-agnostic axes and a fourth combined score. Let the image be divided into an $N \times N$ grid. For each source tile i, let (r_i, c_i) denote its original row and column, and let (r'_i, c'_i) denote its destination after applying the permutation.

4.6.1 Adjacency Destruction Hardness

Main idea. This metric asks how much of the original neighborhood structure survives after the tile permutation. If tiles that were next to each other are no longer neighbors, the image becomes harder for a convolutional model that relies on local structure.

For every original rightward and downward neighboring relation, we test whether the

Metric	Dimension	What it captures
Adjacency destruction hardness	Local topology	Are original neighbors still neighbors?
Global tile displacement	Global geometry	How far did tiles move from their original positions?
Spatial permutation entropy	Permutation disorder	How diverse or irregular are tile movement patterns?
Combined hardness score	Aggregated hardness	Weighted summary of the three normalized axes.

Table 5: Tile permutation hardness metrics used in the analysis.

same two source tiles remain adjacent in the same direction after permutation. The number of such canonical adjacency relations is

$$M = 2N(N-1).$$

If $A_{\text{preserved}}$ is the number of preserved directed adjacencies, adjacency destruction is

$$A = 1 - \frac{A_{\text{preserved}}}{M}.$$

A low value means that much of the original local structure is preserved. A high value means that neighboring tile relations were mostly destroyed.

4.6.2 Global Tile Displacement

Main idea. This metric measures how far tiles move from their original positions. A permutation that only nudges tiles nearby should preserve more global shape than one that sends tiles across the image.

For each tile, we compute its Manhattan displacement in grid coordinates:

$$\delta_i = |r'_i - r_i| + |c'_i - c_i|.$$

The normalized global displacement is

$$D = \frac{\sum_i \delta_i}{D_{\max}},$$

where $D_{\max}$ is the maximum possible total Manhattan displacement for an $N \times N$ grid. A low value indicates that tiles remain close to their original locations. A high value indicates large-scale spatial relocation.

4.6.3 Spatial Permutation Entropy

Main idea. This metric asks whether tile movement follows a simple pattern or many different movement patterns. More varied movement directions and distances mean the permutation is less orderly.

For each tile, we compute

$$\Delta_i = (r'_i - r_i,\ c'_i - c_i)$$

and assign it to a movement bin

$$b_i = (\text{direction}(\Delta_i),\ \text{distanceTier}(\Delta_i)).$$

The direction is one of

$$\{\text{stationary}, N, S, E, W, NE, NW, SE, SW\}.$$

For non-stationary tiles, the normalized Manhattan distance is

$$d_i = \frac{|r'_i - r_i| + |c'_i - c_i|}{2(N-1)}.$$

The distance tier is near when $0 < d_i \leq 1/3$, medium when $1/3 < d_i \leq 2/3$, and far when $2/3 < d_i \leq 1$. Stationary tiles use a single stationary bin. If p_b is the empirical probability of movement bin b, the normalized entropy is

$$E = \frac{-\sum_b p_b \log p_b}{\log(N^2)}.$$

A low value means that movement is structured or concentrated in a small number of movement patterns. A high value means that the permutation contains diverse movement directions and distances, making it more disorderly.

4.6.4 Combined Hardness Score

Main idea. The combined score summarizes the component metrics into one number so permutations can be ranked by an overall notion of difficulty.

The final score is a convex combination of the three normalized components:

$$C = w_a A + w_d D + w_e E,$$

with default weights

$$w_a = 0.5, \quad w_e = 0.3, \quad w_d = 0.2.$$

A low combined score represents an easier permutation with preserved structure, small movement, and low movement disorder. A high combined score represents a harder permutation across these complementary axes.

5 Additional Implementation Details

Table 10 collects the implementation details needed to reproduce the reported run matrix. These details are separated from the main methodology so that the scientific setup remains readable while still documenting the executed configuration.

5.1 Training and Validation Sets by Experiment

All experiments use the same 20,000/5,000 split described in the data overview. Tables 6, 7, and 9 summarize how that split is reused in the three assignment parts.

The curriculum ablations in Part 2 use stage-specific training loaders, but they do not create separate held-out validation splits per stage. Across both curricula, validation remains the same 5,000-image held-out loader for the final target run. Non-curriculum

Part 1 run	**Training set**	**Validation set**
1 tile / none	20,000 unpermuted images repeated under easy, medium, and hard labels.	5,000 unpermuted held-out images.
4×4 permutations	Same 20,000 images with easy, medium, and hard deterministic 4×4 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
7×7 permutations	Same 20,000 images with easy, medium, and hard deterministic 7×7 permutations.	Same 5,000 held-out images with the matching deterministic permutation.
10×10 permutations	Same 20,000 images with easy, medium, and hard deterministic 10×10 permutations.	Same 5,000 held-out images with the matching deterministic permutation.

Table 6: Part 1 training and validation sets. The same split is reused for MobileNetV3-Small, DeiT-Tiny, and gMLP-S16, and each remote run trains for 10 epochs.

Part 2 ablations train for 10 epochs, whereas both curriculum ablations train for 30 epochs. This design should be interpreted cautiously: any difference between curriculum and non-curriculum records is partly confounded by training duration, so the reported results cannot isolate curriculum scheduling from additional optimization time.

6 Evaluation Protocol

Validation accuracy is the main metric throughout the report because the held-out validation set is class-balanced, with equal numbers of cats and dogs. Final validation accuracy reports the value at the end of the configured training run. Best validation accuracy reports the maximum validation accuracy reached during training and is useful for comparing training behavior, but it can be optimistic as a final performance estimate because it selects the best epoch on the validation set.

For Part 3, the joined hardness table keeps the 12 deterministic records in the final configurable-CNN matrix for traceability. The primary pooled correlation analysis collapses the repeated one-tile/no-permutation baseline to one row, giving 10 rows; the original 12-row view and a 9-row non-baseline-only view are retained as sensitivity checks. The correlations should therefore be treated as exploratory descriptors of this controlled matrix rather than definitive statistical estimates. No p-values, confidence intervals, repeated-seed variability estimates, or independent test-set measurements are reported.

7 Results

The results are organized according to the three assignment parts. Section 7.1 reports the frozen-backbone comparison under deterministic tile permutations. Section 7.2 reports the configurable-CNN ablation matrix. Section 7.3 reports the model-agnostic hardness metrics and their relationship to validation accuracy.

All result statements in this section should be read as validation-set findings for one train/validation split, one main seed, frozen pretrained backbones, and a limited deterministic permutation matrix with three presets per grid size. They are not independent test-set estimates, and they do not quantify repeated-seed variability.

7.1 Part 1: Frozen-Backbone Baselines

Table 11 summarizes the frozen-backbone architecture comparison by tile count, and Figure 4 visualizes the same trend across grid sizes. The notebook records the main frozen-backbone runs, the unfrozen control, and the zero-shot control with explicit condition labels so those results can be reported separately.

The main Part 1 analysis compares the three ImageNet-1k pretrained backbones under the same deterministic permutation matrix. If the hardest conditions remain far above chance, one plausible explanation is that tile permutation preserves local texture, color, object-part, and background cues, and ImageNet-trained classifiers are known to rely heavily on texture and other local cues [6, 9]. The notebook includes explicit checks of label order, validation balance, train/validation separation, and the hard-row calculation before drawing conclusions from the controls.

7.2 Part 2: Configurable-CNN Improvement Strategies

Table 12 reports the MobileNetV3-Small CNN ablation summary over the 12 deterministic records, and Figure 5 shows each strategy's delta from its matched baseline.

The Part 2 matrix is designed to compare each ablation against the matched regular baseline at the same grid size and deterministic difficulty label. The reported deltas should be interpreted within the limited deterministic matrix and the shared validation-

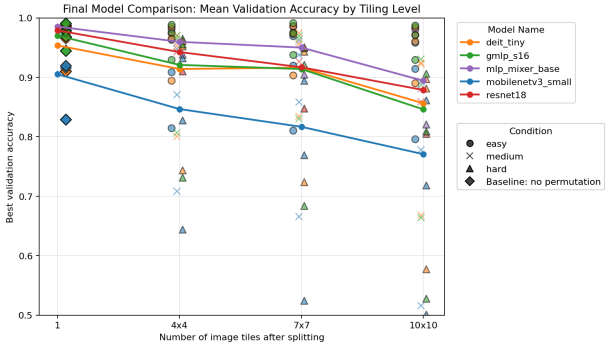

Figure 4: Part 1 mean best validation accuracy as a function of tile count for the three frozen pretrained backbones. Higher tile counts correspond to finer grids and more opportunities for tile-wise spatial disruption; the plotted values average over deterministic difficulty presets at each grid size.

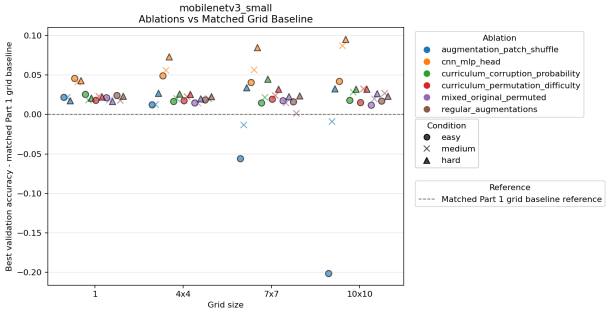

Figure 5: Part 2 best-validation-accuracy delta from the matched CNN baseline. Positive values indicate improvement relative to the same grid size and deterministic permutation preset; negative values indicate that the ablation underperformed the matched baseline.

set selection protocol.

7.3 Part 3: Hardness Metrics

Table 13 reports exploratory correlations between deterministic hardness metrics and MobileNetV3-Small best validation accuracy under the primary one-baseline-row scope. Figures 6–8 visualize the metric values and their relationship to grid size, difficulty label, and accuracy.

Part 3 provides a ranking lens for the deterministic permutations. In the primary one-baseline-row scope, all four hardness metrics were negatively correlated with MobileNetV3-Small best validation accuracy. Adjacency destruction had the largest-magnitude single-metric association, with Pearson correlation -0.9120 and Spearman correlation -0.9423. The retained 12-row full scope gives similar but baseline-weighted correlations, while the 9-row non-baseline-only scope checks that the association is not solely driven by the clean baseline.

8 Discussion

The proposal asks whether a neural network can classify post-permutation images and whether performance depends on tile resolution and permutation choice. The deterministic matrix is designed to test this directly across grid size, difficulty label, and model family. The expected pattern is not merely whether accuracy is above chance, but whether accuracy drops with stronger deterministic disruption under a controlled protocol. The implementation standardizes the backbone comparison to MobileNetV3-Small, DeiT-Tiny, and gMLP-S16 with ImageNet-1k-only timm checkpoints.

This interpretation is intentionally scoped to the measured validation runs. Because the matrix uses only three deterministic presets per grid size and no repeated training seeds or independent test set, the magnitude and ordering of these effects may change under broader sampling or a different evaluation protocol.

The architecture comparison is controlled with respect to pretraining source because all

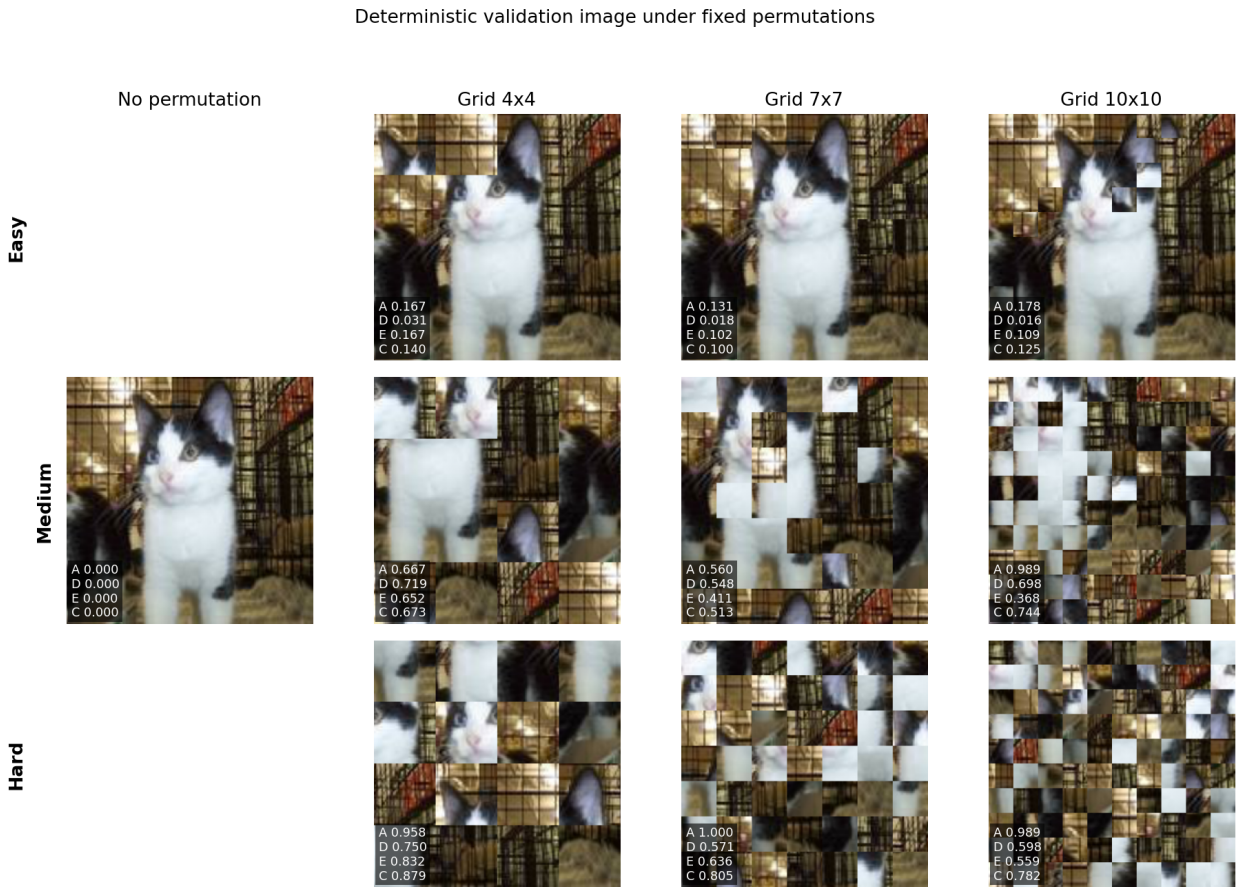

Figure 6: Part 3 example permutations and model-agnostic hardness metrics for the deterministic grid and difficulty settings. The figure is intended to make the easy, medium, and hard presets visually comparable across grid sizes.

three selected checkpoints are ImageNet-1k-only. This does not show that DeiT-Tiny or gMLP-S16 is generally superior for permuted images, nor that attention or token mixing solves permutation hardness. The backbones are frozen, pretrained on natural images, and differ in several ways beyond architecture. A cautious interpretation is that class-discriminative information may remain available after the deterministic tile-wise permutations and can be accessed by pretrained features plus a small classification head in this protocol.

The Part 2 ablations test whether simply adding spatial noise, changing the training distribution, increasing head capacity, or scheduling difficulty improves the target task. Patch shuffle may help only if the additional random shuffling matches useful invariances for the fixed validation permutation. The MLP head tests whether additional trainable capacity at the classifier stage improves the mapping from frozen convolutional features to dog/cat labels.

The hardness metrics address the proposal's third objective: ranking permutations with the same tile count. The negative correlations suggest that geometry-only permutation de-

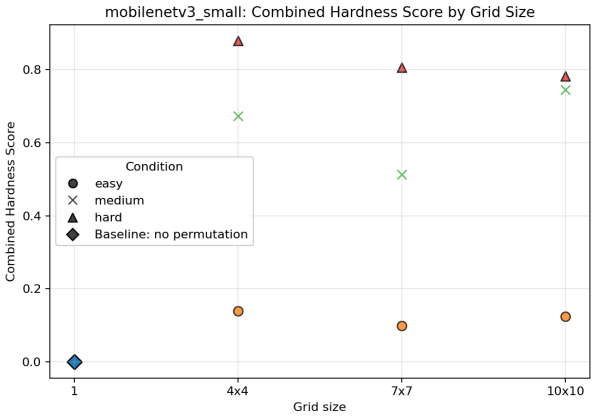

Figure 7: Combined hardness score by grid size and deterministic difficulty label. The score aggregates normalized adjacency destruction, global displacement, and spatial entropy.

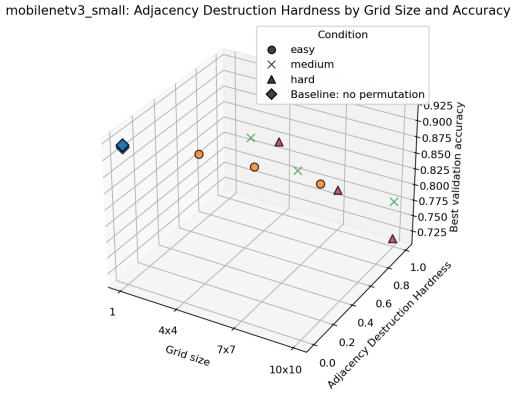

Figure 8: Adjacency destruction, grid size, and MobileNetV3-Small best validation accuracy across the joined deterministic records. Lower accuracy is associated with higher adjacency destruction in this exploratory matrix.

scriptors can capture meaningful variation in MobileNetV3-Small performance within the deterministic matrix. Adjacency destruction was particularly associated with validation accuracy, which is consistent with the intuition that destroying local tile neighborhoods can be difficult for a frozen convolutional model. Because the primary analysis uses only 10 rows after collapsing the repeated clean baseline, it should be viewed as a hypothesis-generating result for future experiments with more permutation samples.

Finally, high accuracy should not be interpreted as evidence that the models learned to undo the permutation. The experiments do not directly distinguish spatial reconstruction, compensation for a fixed mapping, and exploitation of surviving class-discriminative cues. One possible explanation is that local texture, object parts, background statistics, or other cues persist even when global shape is disrupted; the current experiments cannot distinguish this explanation from compensation for the fixed mapping.

9 Limitations

Several limitations constrain the strength and scope of the conclusions. First, no repeated training seeds are reported, so run-to-run variation is not quantified. Second, no independent labeled test set is used, so validation accuracy may overstate final generalization. Best validation accuracy is especially optimistic because it selects the best epoch on the validation set. Third, the architecture comparison is confounded by pretraining, parameter counts, patch sizes, tokenization behavior, and model-specific feature representations. Fourth, the frozen-backbone setup limits conclusions about how fully trainable models would adapt to tile-wise permutations.

The experimental matrix is also limited. Curriculum results are partly confounded by longer training duration, since curriculum ablations train for 30 epochs while non-curriculum ablations train for 10 epochs. The deterministic matrix contains only three per-

mutation presets per grid size, and the primary hardness-correlation analysis uses only 10 rows after collapsing the repeated one-tile baseline. The study does not test full fine-tuning, unseen permutations, transfer across permutations, cryptographic recovery, reconstruction, inversion, or explicit learning of the original layout. Finally, Dogs-versus-Cats may remain classifiable from local texture, object-part, or background cues even when global structure is disrupted. The report therefore does not directly distinguish learning or compensating for a fixed permutation from exploiting remaining class-discriminative cues.

10 Conclusion and Future Work

This report presents a reproducible Dogs-versus-Cats testbed for studying fixed tile-wise image permutations. The implementation uses MobileNetV3-Small, DeiT-Tiny, and gMLP-S16 as the ImageNet-1k-only pretrained backbone set. The experiment matrix is designed to test whether fixed permutations reduce validation accuracy without eliminating class-discriminative signal, and whether the effect differs between convolutional, transformer, and MLP-style vision backbones.

Within the scope of the experiments, these results suggest that pretrained visual representations can preserve useful dog/cat information even when global image layout is disrupted by the tested deterministic tile-wise permutations. They also suggest that local adjacency destruction may be a useful model-agnostic descriptor of permutation difficulty for frozen CNNs in this matrix. The results do not prove that models reconstruct, invert, or explicitly learn the permutation, and they do not establish general robustness to unseen permutations, transfer across permutations, or cryptographic hardness.

Future work should include repeated training seeds, independent test evaluation, full fine-tuning, more permutation samples per grid size, transfer tests between permutation functions, and experiments on datasets where global shape is more necessary for classification. Overall, the study provides preliminary empirical evidence and a reproducible framework for analyzing classification under deterministic spatial permutations, while showing that stronger statistical validation and transfer experiments are needed before drawing broader conclusions about permutation robustness.

Part 2 strategy	**Training set**	**Added training exposure**	**Validation set**
Regular Part 1 baseline	Reused Part 1 configurable-CNN runs.	None beyond the fixed target permutation.	Reused Part 1 held-out validation accuracy.
Patch shuffle	Same 20,000 training images and target permutation.	No appended files; each batch of size B produces B shuffled training views, for 20,000 shuffled views per epoch.	Same 5,000 held-out images with the target permutation only.
Regular augmentations	Same 20,000 training images and target permutation.	Standard stochastic training transforms produce one augmented view per example per epoch.	Same 5,000 held-out images with the target permutation only.
Mixed original/permuted	Same 20,000 training images.	Each training view is sampled as original or target-permuted, using $p_{\text{original}} = 0.5$.	Same 5,000 held-out images with the target permutation only.
CNN MLP head	Same 20,000 training images and target permutation.	No new images; the linear classifier is replaced by a small MLP head.	Same 5,000 held-out images with the target permutation.
Corruption-probability curriculum	Same 20,000 training images at every stage.	No appended files; stage probabilities 0.1, 0.3, 0.5, and 0.8 give expected permuted counts of 2,000, 6,000, 10,000, and 16,000 views per stage epoch.	One fixed target validation loader; no separate validation set per stage.
Permutation-difficulty curriculum	Same 20,000 training images at every stage.	No appended files; each stage processes 20,000 views per epoch at its scheduled difficulty: original, 2×2, 3×3, 4×4, and for 7×7 or 10×10 targets an additional final target-grid stage.	One fixed final-target validation loader; no separate validation set per stage.

Table 7: Part 2 training and validation sets by ablation. The final output matrix uses the configured CNN backbone, defaulting to MobileNetV3-Small, with 4 grid settings crossed with easy, medium, and hard deterministic permutation records, for 12 records per ablation.

Part 2 strategy	**Stages**	**Experiment sequence**	**Dataset increase / per-epoch views**
Regular Part 1 baseline	1	Standard training on the target permutation.	No increase: 20,000 training examples per epoch.
Patch shuffle	1	Standard training, with random patch shuffling applied inside training batches.	0 stored increase; B shuffled views per batch and 20,000 per epoch.
Regular augmentations	1	Standard training, with generic image augmentations in the training transform.	0 stored increase; one stochastic view per example per epoch.
Mixed original/permuted	1	Standard training, sampling clean and target-permuted views with equal probability.	0 stored increase; expected 10,000 clean and 10,000 permuted views per epoch.
CNN MLP head	1	Standard target-permutation training, replacing the linear head with an MLP head.	No increase: the examples are unchanged, only the trainable head changes.
Corruption-probability curriculum	4	Permutation probability stages: 0.1, 0.3, 0.5, 0.8.	0 stored increase; expected permuted views per epoch are 2,000, 6,000, 10,000, and 16,000 by stage.
Permutation-difficulty curriculum	1, 4, or 5	Baseline target: original only. Tiled targets: original, 2×2, 3×3, 4×4, and the final target grid when larger than 4×4, using the same easy/medium/hard function as the target record.	0 stored increase; 20,000 scheduled-difficulty views per stage epoch.

Table 8: Part 2 ablation experiment design. Non-curriculum ablations train for 10 epochs; both curriculum ablations train for 30 epochs. None of the ablations duplicate files on disk.

Part 3 analysis	**Training set**	**Validation source**
Hardness metric computation	Not applicable; Part 3 does not train a model.	Reuses Part 1 configurable-CNN validation accuracies.
Metric-accuracy join	Not applicable; only saved CSV outputs are joined.	Matches each metric row to the corresponding Part 1 permutation ID.
Correlation analysis	Not applicable; no parameters are updated.	Computes correlations against the joined held-out validation accuracies.

Table 9: Part 3 data usage. Part 3 analyzes saved permutations and validation accuracies rather than creating new train or validation splits.

Item	Value to report
Dataset source	Kaggle Dogs vs Cats labeled training images; the supervised experiments use the labeled 25,000-image portion.
Train/validation split procedure	Stratified class-balanced split; validation fraction 0.2; test fraction 0.0; split seed 42.
Image preprocessing	Resize to the smallest square size divisible by the grid side length, convert to RGB float tensor, and normalize with ImageNet mean and standard deviation; timm models are center-cropped back to 224 after tile-compatible resizing when required.
Tile transform details	Tiles are square non-overlapping tensor blocks after resizing; the permutation copies source tiles into output positions with no interpolation during tile rearrangement; preset parameters are listed in Table 1.
Optimization	AdamW; learning rate 3×10^{-4}; weight decay 1×10^{-4}; cross-entropy loss; batch size 64 and 4 dataloader workers in the Colab runs; no learning-rate scheduler is used.
Training duration	Remote Part 1 runs train for 10 epochs; non-curriculum Part 2 ablations train for 10 epochs; curriculum Part 2 ablations train for 30 epochs.
Model configuration	Part 1 models use timm ImageNet-1k checkpoints: `mobilenetv3_small_100`, `deit_tiny_patch16_224.fb_in1k`, and `gmlp_s16_224.ra3_in1k`. The main Part 1 baseline freezes backbones and trains only binary heads; the controls either unfreeze the pretrained binary-head model or preserve the native ImageNet-1k head for zero-shot dog/cat scoring. The configurable-CNN MLP head is Linear(in_features,256), BatchNorm1d, ReLU, Dropout(0.3), Linear(256,2).
Augmentation and curricula	Patch shuffle randomly permutes within-image patches during training; regular augmentation uses random rotation, color jitter, resize, tensor conversion, and normalization; mixed-input training uses $p_{\text{original}} = 0.5$; curriculum definitions are given in Table 8.
Compute environment	The executed environment records Python 3.12.13, numpy 2.0.2, pandas 2.2.2, scipy 1.16.3, scikit-learn 1.6.1, torch 2.11.0+cu128, torchvision 0.26.0+cu128, timm 1.0.27, PyYAML 6.0.3, and an NVIDIA L4 GPU. Seed 42 is used, but deterministic GPU kernels are not enforced.
Artifacts	The saved outputs include experiment scripts, raw and aggregated CSV files, joined hardness metrics, figures, and checkpoints for the reported run matrix.

Table 10: Reproducibility checklist for the reported experiment matrix.

Model	1 tile best	16 tiles best	49 tiles best	100 tiles best
MobileNetV3-Small	91.82%	86.93%	84.92%	80.36%
DeiT-Tiny	98.75%	97.23%	97.08%	92.97%
gMLP-S16	99.09%	97.16%	96.11%	89.47%

Table 11: Part 1 mean best validation accuracy by model and tile count, aggregated across the easy, medium, and hard deterministic records at each grid size.

Part 2 setting	Mean final acc.	Mean best acc.	Mean best delta vs. baseline
Regular Part 1 baseline	pending	pending	0.00 pp
Patch shuffle	pending	pending	pending
Regular augmentations	pending	pending	pending
Mixed original/permuted	pending	pending	pending
CNN MLP head	pending	pending	pending
Corruption-probability curriculum	pending	pending	pending
Permutation-difficulty curriculum	pending	pending	pending

Table 12: Part 2 MobileNetV3-Small CNN ablation summary over the 12 deterministic records. "Mean final acc." is the validation accuracy at the final epoch of the corresponding strategy. "Mean best acc." is the best validation accuracy reached during that same strategy's training run.

Hardness metric	**Pearson**	**Spearman**	**N**
Global displacement	-0.7463	-0.6970	10
Adjacency destruction	-0.9120	-0.9423	10
Spatial entropy	-0.6650	-0.7212	10
Combined hardness	-0.8481	-0.8909	10

Table 13: Part 3 exploratory correlations between MobileNetV3-Small best validation accuracy and permutation metrics after collapsing the repeated one-tile/no-permutation baseline to one row. Negative values indicate that larger hardness scores are associated with lower validation accuracy in this matrix.

References

[1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *International Conference on Machine Learning*, pages 41–48, 2009. https://doi.org/10.1145/1553374.1553380.

[2] E. Boyle, J. Holmgren, and M. Weiss. Permuted puzzles and cryptographic hardness. In *Theory of Cryptography*, pages 465–493, 2019. https://doi.org/10.1007/978-3-030-36033-7_18.

[3] C. Doersch, A. Gupta, and A. A. Efros. Unsupervised visual representation learning by context prediction. In *IEEE International Conference on Computer Vision*, pages 1422–1430, 2015. https://doi.org/10.1109/ICCV.2015.167.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. https://openreview.net/forum?id=YicbFdNTTy.

[5] S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *International Conference on Learning Representations*, 2018. https://openreview.net/forum?id=S1v4N2l0-.

[6] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*, 2019. https://openreview.net/forum?id=Bygh9j09KX.

[7] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. https://www.deeplearningbook.org/.

[8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. https://doi.org/10.1109/CVPR.2016.90.

[9] K. L. Hermann, T. Chen, and S. Kornblith. The origins and prevalence of texture bias in convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2020. https://proceedings.neurips.cc/paper/2020/hash/db5f9f42a7157abe65bb145000b5871a-Abstract.html.

[10] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam. Searching for MobileNetV3. In *IEEE/CVF International Conference on Computer Vision*, pages 1314–1324, 2019. https://doi.org/10.1109/ICCV.2019.00140.

[11] H. Liu, Z. Dai, D. R. So, and Q. V. Le. Pay attention to MLPs. In *Advances in Neural Information Processing Systems*, 2021. https://proceedings.neurips.cc/paper/2021/hash/4cc05b35c2f937c5bd9e7d41d3686fff-Abstract.html.

[12] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *European Conference on Computer Vision*, pages 69–84, 2016. https://doi.org/10.1007/978-3-319-46466-4_5.

[13] C. Shorten and T. M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019. https://doi.org/10.1186/s40537-019-0197-0.

[14] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit, M. Lucic, and A. Dosovitskiy. MLP-Mixer: An all-MLP architecture for vision. In *Advances in Neural Information Processing Systems*, 2021. https://proceedings.neurips.cc/paper/2021/hash/cba0a4ee5ccd02fda0fe3f9a3e7b89fe-Abstract.html.

[15] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers and distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021. https://proceedings.mlr.press/v139/touvron21a.html.